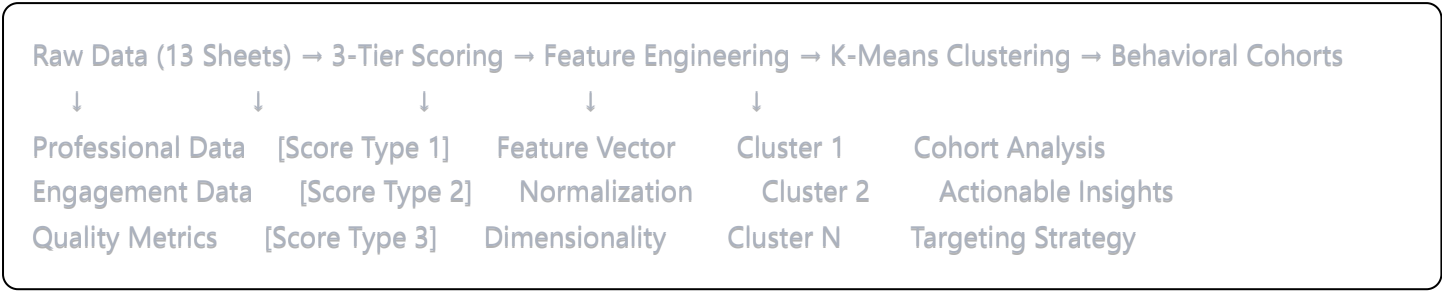


3-Tier Scoring + K-Means Clustering System

Process Flow Overview



1. Three-Tier Scoring System

Tier 1: Persona Scoring (Text + Keyword Analysis)

```
python
```

```

def calculate_tier1_persona_scores(doctor_record):
    """Keyword-based persona scoring from text fields"""

    persona_scores = {}

    # For each of the 7 persona buckets
    personas = ['academic_research', 'incentive_focused', 'patient_focused',
                'technology_focused', 'recognition_focused', 'community_focused', 'bucket7']

    for persona in personas:
        # Text fields contributing to this persona
        text_fields = get_persona_text_fields(persona)

        total_score = 0
        field_weights = get_field_weights(persona)

        for field, weight in field_weights.items():
            if field in text_fields and doctor_record.get(field):
                field_score = calculate_keyword_match_score(
                    text=doctor_record[field],
                    keyword_dict=PERSONA_KEYWORDS[persona],
                    dynamic_weights=True
                )
                total_score += field_score * weight

        persona_scores[f'{persona}_score'] = min(total_score, 100)

    return persona_scores

# Example output:
# {
#   'academic_research_score': 85,
#   'patient_focused_score': 67,
#   'technology_focused_score': 23,
#   ...
# }

```

Tier 2: Engagement Performance Scoring (Mathematical)

python

```

def calculate_tier2_engagement_scores(engagement_record):
    """Mathematical scoring from engagement performance data"""

    engagement_scores = {}

    # Email Engagement Score
    email_open_rate = engagement_record.get('HCP_Email_Open_Rate', 0)
    email_click_rate = engagement_record.get('HCP_Email_Click_Rate', 0)

    # Weighted email score (open rate 40%, click rate 60%)
    engagement_scores['email_engagement_score'] = (email_open_rate * 0.4) + (email_click_rate * 0.6)

    # WhatsApp Engagement Score
    wa_read_rate = engagement_record.get('HCP_WhatsApp_Read_Rate', 0)
    wa_click_rate = engagement_record.get('HCP_WhatsApp_Click_Rate', 0)

    engagement_scores['whatsapp_engagement_score'] = (wa_read_rate * 0.4) + (wa_click_rate * 0.6)

    # Call Performance Score
    call_productive_rate = engagement_record.get('HCP_Call_Productive_Rate', 0)
    avg_call_duration = engagement_record.get('Average_Duration_Connected_Calls', 0)

    # Normalize call duration (assume 0-60 minutes scale to 0-100)
    normalized_duration = min((avg_call_duration / 60) * 100, 100)

    engagement_scores['call_performance_score'] = (call_productive_rate * 0.7) + (normalized_duration * 0.3)

    # Overall Digital Responsiveness Score
    engagement_scores['overall_responsiveness_score'] = (
        engagement_scores['email_engagement_score'] * 0.3 +
        engagement_scores['whatsapp_engagement_score'] * 0.4 +
        engagement_scores['call_performance_score'] * 0.3
    )

    # Channel Preference Indicator
    engagement_scores['preferred_channel'] = determine_preferred_channel(engagement_scores)

    return engagement_scores

def determine_preferred_channel(engagement_scores):
    """Determine doctor's preferred communication channel"""

    scores = {
        'email': engagement_scores['email_engagement_score'],
        'whatsapp': engagement_scores['whatsapp_engagement_score'],
        'call': engagement_scores['call_performance_score']
    }

```

```
}
```

```
return max(scores, key=scores.get)
```

Tier 3: Professional Authority & Quality Scoring (Boolean + Numeric)

```
python
```

```

def calculate_tier3_authority_scores(doctor_record):
    """Professional authority and data quality scoring"""

    authority_scores = {}

    # Research Authority Score
    research_indicators = {
        'has_publications': bool(doctor_record.get('publication_name')),
        'has_h_index': bool(doctor_record.get('h_index', 0) > 0),
        'has_clinical_trials': bool(doctor_record.get('trial_id')),
        'has_awards': bool(doctor_record.get('award_name')),
        'multiple_publications': (doctor_record.get('publication_count', 0) > 3)
    }

    # Weighted research authority
    research_weights = {'has_publications': 20, 'has_h_index': 25, 'has_clinical_trials': 30,
                        'has_awards': 15, 'multiple_publications': 10}

    authority_scores['research_authority_score'] = sum(
        research_weights[indicator] for indicator, present in research_indicators.items() if present
    )

    # Digital Presence Score
    digital_indicators = {
        'has_social_media': bool(doctor_record.get('sm_channel')),
        'has_healthcare_platform': bool(doctor_record.get('healthcare_platforms')),
        'high_social_following': (doctor_record.get('sm_followers', 0) > 1000),
        'multiple_platforms': bool(count_digital_platforms(doctor_record) > 2)
    }

    digital_weights = {'has_social_media': 25, 'has_healthcare_platform': 30,
                       'high_social_following': 25, 'multiple_platforms': 20}

    authority_scores['digital_authority_score'] = sum(
        digital_weights[indicator] for indicator, present in digital_indicators.items() if present
    )

    # Professional Network Score
    network_score = 0
    network_score += min(doctor_record.get('association_count', 0) * 10, 40) # Cap at 40
    network_score += min(doctor_record.get('academic_affiliation_count', 0) * 15, 30) # Cap at 30
    network_score += min(doctor_record.get('years_of_exp', 0) * 2, 30) # Cap at 30

    authority_scores['professional_network_score'] = network_score

    # Data Quality Score (from previous data management system)

```

```
authority_scores['data_quality_score'] = doctor_record.get('overall_quality_score', 50)
```

```
return authority_scores
```

2. Feature Engineering for Clustering

Feature Vector Creation

```
python
```

```

def create_clustering_feature_vector(doctor_record):
    """Create standardized feature vector for K-means clustering"""

    # Get all three tiers of scores
    tier1_scores = calculate_tier1_persona_scores(doctor_record)
    tier2_scores = calculate_tier2_engagement_scores(doctor_record)
    tier3_scores = calculate_tier3_authority_scores(doctor_record)

    # Combine all scores into feature vector
    feature_vector = {
        # Tier 1: Persona Scores (7 features)
        'academic_score': tier1_scores['academic_research_score'],
        'incentive_score': tier1_scores['incentive_focused_score'],
        'patient_score': tier1_scores['patient_focused_score'],
        'technology_score': tier1_scores['technology_focused_score'],
        'recognition_score': tier1_scores['recognition_focused_score'],
        'community_score': tier1_scores['community_focused_score'],
        'bucket7_score': tier1_scores['bucket7_score'],

        # Tier 2: Engagement Scores (4 features)
        'email_engagement': tier2_scores['email_engagement_score'],
        'whatsapp_engagement': tier2_scores['whatsapp_engagement_score'],
        'call_performance': tier2_scores['call_performance_score'],
        'overall_responsiveness': tier2_scores['overall_responsiveness_score'],

        # Tier 3: Authority Scores (4 features)
        'research_authority': tier3_scores['research_authority_score'],
        'digital_authority': tier3_scores['digital_authority_score'],
        'professional_network': tier3_scores['professional_network_score'],
        'data_quality': tier3_scores['data_quality_score'],

        # Derived Features (3 features)
        'persona_clarity': calculate_persona_clarity(tier1_scores),
        'engagement_consistency': calculate_engagement_consistency(tier2_scores),
        'overall_influence': calculate_overall_influence_score(tier3_scores)
    }

    return feature_vector

def calculate_persona_clarity(persona_scores):
    """Measure how clearly defined a doctor's persona is"""
    scores = list(persona_scores.values())
    scores.sort(reverse=True)

    # High clarity = large gap between top 2 scores
    if len(scores) >= 2:

```

```
        return min((scores[0] - scores[1]) * 2, 100)
    return 0

def calculate_engagement_consistency(engagement_scores):
    """Measure consistency across engagement channels"""
    channel_scores = [
        engagement_scores['email_engagement_score'],
        engagement_scores['whatsapp_engagement_score'],
        engagement_scores['call_performance_score']
    ]

    # Low standard deviation = high consistency
    import numpy as np
    consistency = 100 - (np.std(channel_scores) * 3) # Scale factor
    return max(0, consistency)
```

Feature Normalization and Scaling

python


```

def normalize_features_for_clustering(feature_vectors):
    """Normalize all features to 0-1 scale for K-means"""

    import pandas as pd
    from sklearn.preprocessing import StandardScaler, MinMaxScaler

    # Convert to DataFrame
    df = pd.DataFrame(feature_vectors)

    # Handle different scaling needs
    scaler_standard = StandardScaler() # For normally distributed features
    scaler_minmax = MinMaxScaler()     # For bounded features (0-100)

    # Apply appropriate scaling
    normalized_features = {}

    # Persona scores (already 0-100, use MinMax)
    persona_cols = ['academic_score', 'incentive_score', 'patient_score',
                    'technology_score', 'recognition_score', 'community_score', 'bucket7_score']
    normalized_features.update(dict(zip(persona_cols, scaler_minmax.fit_transform(df[persona_cols]).T)))

    # Engagement scores (rates/percentages, use MinMax)
    engagement_cols = ['email_engagement', 'whatsapp_engagement', 'call_performance', 'overall_responsiveness']
    normalized_features.update(dict(zip(engagement_cols, scaler_minmax.fit_transform(df[engagement_cols]).T)))

    # Authority scores (mixed distributions, use StandardScaler)
    authority_cols = ['research_authority', 'digital_authority', 'professional_network', 'data_quality']
    normalized_features.update(dict(zip(authority_cols, scaler_standard.fit_transform(df[authority_cols]).T)))

    # Derived features (use StandardScaler)
    derived_cols = ['persona_clarity', 'engagement_consistency', 'overall_influence']
    normalized_features.update(dict(zip(derived_cols, scaler_standard.fit_transform(df[derived_cols]).T)))

    return pd.DataFrame(normalized_features), scaler_minmax, scaler_standard

```

3. K-Means Clustering Implementation

Optimal Cluster Number Detection

python

```

def find_optimal_clusters(feature_matrix, max_clusters=20):
    """Find optimal number of clusters using multiple methods"""

    from sklearn.cluster import KMeans
    from sklearn.metrics import silhouette_score
    import numpy as np
    import matplotlib.pyplot as plt

    # Method 1: Elbow Method
    inertias = []
    K_range = range(2, max_clusters + 1)

    for k in K_range:
        kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)
        kmeans.fit(feature_matrix)
        inertias.append(kmeans.inertia_)

    # Find elbow point
    elbow_k = find_elbow_point(K_range, inertias)

    # Method 2: Silhouette Analysis
    silhouette_scores = []
    for k in K_range:
        kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)
        cluster_labels = kmeans.fit_predict(feature_matrix)
        silhouette_avg = silhouette_score(feature_matrix, cluster_labels)
        silhouette_scores.append(silhouette_avg)

    silhouette_optimal_k = K_range[np.argmax(silhouette_scores)]

    # Method 3: Gap Statistic (simplified)
    gap_optimal_k = calculate_gap_statistic(feature_matrix, K_range)

    # Combine recommendations
    recommendations = {
        'elbow_method': elbow_k,
        'silhouette_method': silhouette_optimal_k,
        'gap_statistic': gap_optimal_k,
        'suggested_k': most_common([elbow_k, silhouette_optimal_k, gap_optimal_k])
    }

    return recommendations

def perform_kmeans_clustering(feature_matrix, n_clusters):
    """Perform K-means clustering with detailed analysis"""

```

```

from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score, calinski_harabasz_score

# Perform clustering
kmeans = KMeans(n_clusters=n_clusters, random_state=42, n_init=10, max_iter=300)
cluster_labels = kmeans.fit_predict(feature_matrix)

# Calculate quality metrics
silhouette_avg = silhouette_score(feature_matrix, cluster_labels)
calinski_harabasz = calinski_harabasz_score(feature_matrix, cluster_labels)

# Get cluster centers
cluster_centers = kmeans.cluster_centers_

# Calculate cluster statistics
cluster_stats = {}
for i in range(n_clusters):
    cluster_mask = cluster_labels == i
    cluster_size = np.sum(cluster_mask)
    cluster_stats[i] = {
        'size': cluster_size,
        'percentage': (cluster_size / len(cluster_labels)) * 100,
        'center': cluster_centers[i],
        'members': np.where(cluster_mask)[0]
    }

return {
    'model': kmeans,
    'labels': cluster_labels,
    'centers': cluster_centers,
    'silhouette_score': silhouette_avg,
    'calinski_harabasz_score': calinski_harabasz,
    'cluster_stats': cluster_stats,
    'n_clusters': n_clusters
}

```

4. Behavioral Cohort Analysis

Cluster Characterization

python

```

def characterize_clusters(clustering_results, original_data, feature_names):
    """Analyze and characterize each cluster"""

    cluster_profiles = {}

    for cluster_id in range(clustering_results['n_clusters']):
        cluster_mask = clustering_results['labels'] == cluster_id
        cluster_members = original_data[cluster_mask]

        # Calculate mean values for each feature in this cluster
        cluster_means = cluster_members.mean()

        # Identify defining characteristics (features significantly above/below global mean)
        global_means = original_data.mean()
        feature_deviations = {}

        for feature in feature_names:
            cluster_mean = cluster_means[feature]
            global_mean = global_means[feature]
            deviation = ((cluster_mean - global_mean) / global_mean) * 100

            if abs(deviation) > 20: # Significant deviation threshold
                feature_deviations[feature] = {
                    'cluster_mean': cluster_mean,
                    'global_mean': global_mean,
                    'deviation_percent': deviation,
                    'significance': 'high' if abs(deviation) > 50 else 'moderate'
                }

        # Generate cluster description
        cluster_description = generate_cluster_description(feature_deviations)

        # Sample members for validation
        sample_members = cluster_members.sample(min(10, len(cluster_members)))

        cluster_profiles[cluster_id] = {
            'size': len(cluster_members),
            'percentage': (len(cluster_members) / len(original_data)) * 100,
            'defining_features': feature_deviations,
            'description': cluster_description,
            'sample_members': sample_members.to_dict('records'),
            'centroid_profile': cluster_means.to_dict()
        }

    return cluster_profiles

```

```

def generate_cluster_description(feature_deviations):
    """Generate human-readable cluster descriptions"""

    high_features = []
    low_features = []

    feature_mapping = {
        'academic_score': 'Academic/Research Focus',
        'patient_score': 'Patient-Centric Approach',
        'technology_score': 'Technology Adoption',
        'email_engagement': 'Email Responsiveness',
        'whatsapp_engagement': 'WhatsApp Engagement',
        'call_performance': 'Phone Communication',
        'research_authority': 'Research Credibility',
        'digital_authority': 'Digital Presence'
    }

    for feature, stats in feature_deviations.items():
        feature_name = feature_mapping.get(feature, feature.replace('_', ' ').title())

        if stats['deviation_percent'] > 20:
            high_features.append(feature_name)
        elif stats['deviation_percent'] < -20:
            low_features.append(feature_name)

    description_parts = []

    if high_features:
        description_parts.append(f"High: {' '.join(high_features[:3])}")

    if low_features:
        description_parts.append(f"Low: {' '.join(low_features[:3])}")

    return " | ".join(description_parts)

```

Cohort Business Value Analysis

python

```

def analyze_cohort_business_value(cluster_profiles, engagement_data):
    """Analyze business value and targeting potential of each cohort"""

    cohort_analysis = {}

    for cluster_id, profile in cluster_profiles.items():

        # Engagement success analysis
        cluster_engagement = engagement_data[engagement_data['cluster'] == cluster_id]

        engagement_success = {
            'avg_email_open_rate': cluster_engagement['HCP_Email_Open_Rate'].mean(),
            'avg_call_success_rate': cluster_engagement['HCP_Call_Productive_Rate'].mean(),
            'preferred_channel': cluster_engagement['preferred_channel'].mode().iloc[0],
            'response_consistency': cluster_engagement['engagement_consistency'].mean()
        }

        # Targeting recommendations
        targeting_strategy = generate_targeting_strategy(profile, engagement_success)

        # Business value scoring
        business_value = calculate_business_value_score(profile, engagement_success)

        cohort_analysis[cluster_id] = {
            'cluster_name': f"Cohort {cluster_id}",
            'suggested_name': generate_cohort_name(profile),
            'business_value_score': business_value,
            'targeting_strategy': targeting_strategy,
            'engagement_profile': engagement_success,
            'size': profile['size'],
            'key_characteristics': profile['description'],
            'priority_level': determine_priority_level(business_value, profile['size'])
        }

    return cohort_analysis

def generate_cohort_name(profile):
    """Generate meaningful names for cohorts"""

    defining_features = profile['defining_features']

    # Name based on strongest characteristics
    names = []

    if 'academic_score' in defining_features and defining_features['academic_score']['deviation_percent'] > 40:
        names.append("Research-Driven")

```

```
elif 'patient_score' in defining_features and defining_features['patient_score']['deviation_percent'] > 40:
    names.append("Patient-Centric")
elif 'technology_score' in defining_features and defining_features['technology_score']['deviation_percent'] > 40:
    names.append("Tech-Savvy")

if 'digital_authority' in defining_features and defining_features['digital_authority']['deviation_percent'] > 30:
    names.append("Digital Influencers")
elif 'research_authority' in defining_features and defining_features['research_authority']['deviation_percent'] > 30:
    names.append("Academic Leaders")

if 'email_engagement' in defining_features and defining_features['email_engagement']['deviation_percent'] > 30:
    names.append("Email Responsive")
elif 'call_performance' in defining_features and defining_features['call_performance']['deviation_percent'] > 30:
    names.append("Phone Preferred")

return " | ".join(names[:2]) if names else f"Mixed Profile"
```

5. Output Structure

Cohort Analysis Results

python

Final output structure for each doctor

doctor_cohort_profile = {

 'UIN': 'D001',

 'full_name': 'Dr. John Smith',

Tier 1 Scores

 'persona_scores': {

 'academic_research': 85,

 'patient_focused': 67,

 'technology_focused': 23,

... other personas

 },

Tier 2 Scores

 'engagement_scores': {

 'email_engagement': 76,

 'whatsapp_engagement': 45,

 'call_performance': 82,

 'overall_responsiveness': 71,

 'preferred_channel': 'call'

 },

Tier 3 Scores

 'autority_scores': {

 'research_authority': 78,

 'digital_authority': 34,

 'professional_network': 56,

 'data_quality': 85

 },

Clustering Results

 'cluster_assignment': {

 'cluster_id': 2,

 'cluster_name': 'Research-Driven | Email Responsive',

 'confidence': 0.78,

 'distance_to_centroid': 1.23

 },

Cohort Analysis

 'cohort_profile': {

 'cohort_name': 'Academic Email Responders',

 'cohort_size': 234,

 'business_value_score': 82,

 'targeting_strategy': 'Email-first with research content',

 'priority_level': 'High'



This system will create highly actionable doctor segments based on personality, engagement behavior, and professional authority - perfect for personalized marketing and sales strategies!